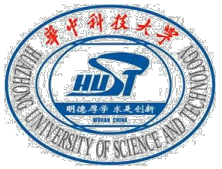
8051 汇编指令与程序设计



51 指令类型

- 寻址方式
- 数据传送与交换指令
- 算术运算、逻辑运算指令
- 控制转移指令
- 位操作指令

寻址方式

- **MCS-51** 单片机汇编语言指令格式：

  操作符　　目的操作数，源操作数

- 操作符指明该指令完成什么操作，源操作数是指明该指令的操作对象，目的操作数存放结果
- 指令中操作数的提供方式称为寻址方式

寻址方式

- 指令中的常用符号：

Rn: n=(0～7），表示当前工作寄存器 R0～R7 中的一个
Ri: i=(0、1），代表 R0 和 R1 寄存器中的一个，用作间接寻址寄存器
dir：8 位直接字节地址（片内 RAM 和 SFR ）
#data: 8 位立即数，即 8 位常数。可以为 2 进制 (B)、 8 进制 (O)、 10 进制 (D)、 16 进制 (H)、字符 (‘’)
#data16: 表示 16 位立即数，即 16 位常数，取值范围为 #0000H～#0FFFFH
addr16：表示 16 位地址
addr11：表示 11 位地址
rel：相对偏移量（为一字节补码）用于相对转移指令中
bit：位地址，在位地址空间中。
$: 表示当前指令的地址。

寻址方式——立即寻址

- 立即寻址：指令中直接给出操作数的寻址方式
- 立即数用一个前面加 “ #” 号
- 立即寻址中的数，称为立即数
- 立即数可以是 8 位数 (#data ， 如 #30H) 或 16 位数 (#data16 ， 如 #2052H) 表示

例： MOV A ， #30H

寻址方式——直接寻址

- 直接寻址：在指令中直接给出操作数的地址
- 直接寻址对象：
  - 内部数据存贮器：使用其地址。
  - 特殊功能寄存器：可使用其地址，也可使用寄存器名

例 1： MOV A， 40H ；

机器码 E540

例 2： MOV 40H， 41H；机器码为 854140

内部 RAM（41H）→（40H）

例 3： MOV P0， #45H ； 45H→P0，（40H）=（41H）

P0 为直接寻址的 SFR，其地址为 80H，

机器码为 758045

寻址方式——寄存器寻址

- 寄存器寻址：操作数存放在寄存器中
- 寄存器寻址对象仅限： A ， B ， DPTR ， R0 ～ R7
- B 仅在乘除法指令可作寄存器寻址，其他指令为直接寻址。
- A 可寄存器寻址也可直接寻址，直接寻址时写作 ACC
- 机器码中出现字节地址的是直接寻址

例 1： MOV A ， R0 ； R0→A ， A 、 R0 均为寄存器寻址，机器码 E8

例 2： MUL AB ； A*B→BA ， A 、 B 为寄存器寻址，机器码 A4

例 3： MOV B ， R0 ； R0→B ， R0 为寄存器寻址， B 为直接寻址

机器码 88F0 ，其中 F0 为 B 的字节地址

例 4： PUSH ACC ； A 的内容压入堆栈，机器码 C0E0

寻址方式——寄存器间接寻址

- **寄存器间接寻址：**操作数存放在以寄存器内容为地址的存储单元中
- 间接寻址的寄存器限于： **R0 、 R1 、 DPTR**

```
MOV  R0,#20H
MOV  @R0 ，  A        ；  A→(20H)  地址的内部 RAM

MOVX   A ，  @R1      ；外部 RAM （地址为 P2 R1 ）的内容
→ A
```

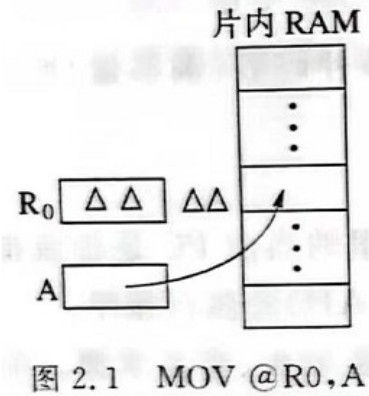


图 2.1　MOV @R0,A

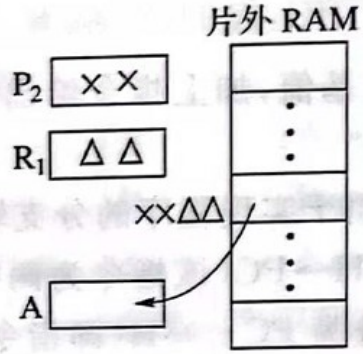


图 2.2　MOVX A,@R1

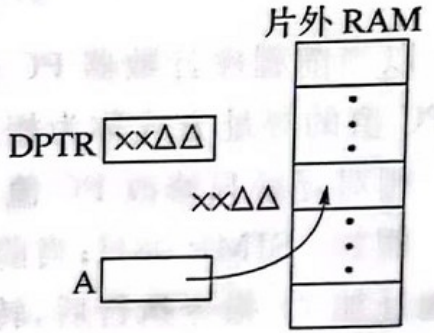


图 2.3　MOVX @DPTR,A

8 位 R1 无法寻址 64k

P2 口输出高 8 位地址信号

寻址方式——变址寻址

- 变址寻址：以 **DPTR** 或 **PC** 寄存器内容为基地址，和 **A** 的内容相加形成操作数的地址
- 只能对程序存储器的数据进行寻址

例如： MOVC A, @A+DPTR
MOVC A, @A+PC

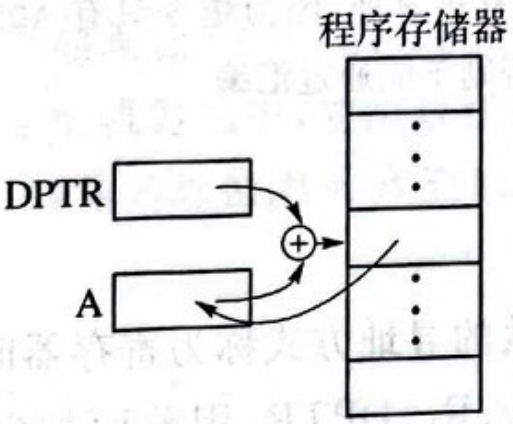


图 2.4 MOVC A,@A+DPTR

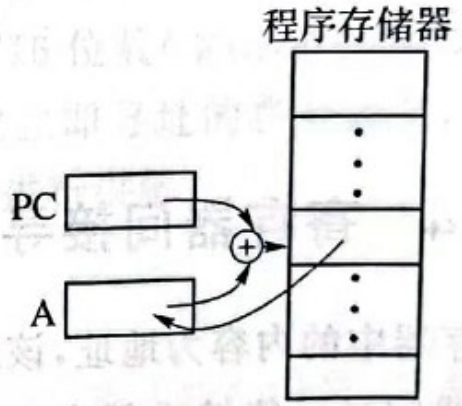


图 2.5 MOVC A,@A+PC

寻址方式——相对寻址

- **相对寻址：**当前程序计数器 **PC** 的值为基础，加上指令给出的一字节补码数（偏移量），形成目标地址
- 相对地址只修改 **PC** 值，主要用于程序的分支转移

例如： SJMP 08H

当前 PC+08H -》 PC， 该指令 2 个字节，指令执行后， PC 值为“指令地址 +0AH”

寻址方式——位寻址

- 位寻址：对片内 **RAM** 中 **20H** ～ **2FH** 中的 **128** 个位地址及 **SFR** 中的可位寻址的位地址寻址

例如： **MOV C，20H**；**20H** 位的内容赋给 **CY** 标志位，**C** 称为位累加器。

MOV A，20H ；字节寻址，将内部 **RAM** 中 **20H** 单元中的值赋值给 **A** 。

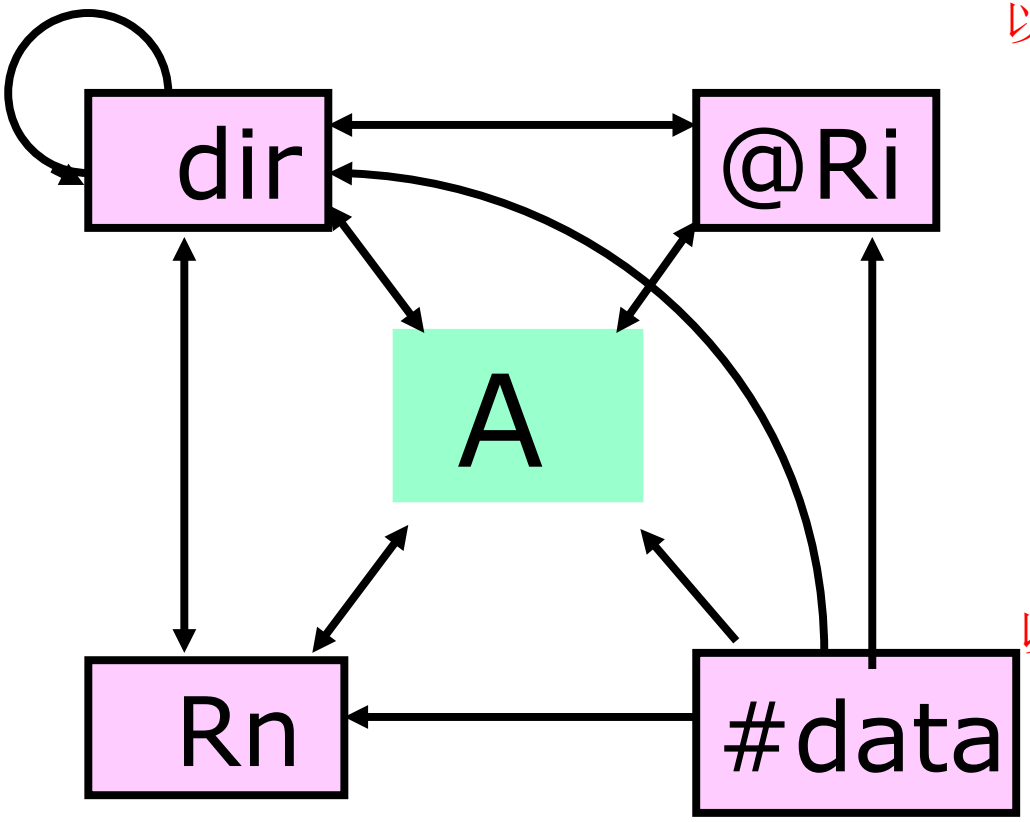
以上两条指令均为寻址，究竟是位寻址还是字节寻址，根据两操作数类型一致的原则，由另一个操作数决定。

CPU 的指令系统

- 寻址方式
- 数据传送与交换指令
- 算术运算、逻辑运算指令
- 控制转移指令
- 位操作指令

数据传送——内部存储器传送 MOV

- 内部 **RAM** 、 **SFR** 之间用 **MOV**



MOV A { Rn, #data, dir, @Ri }

以 A 为目的操作数

MOV dir { A, Rn, @Ri, #data, dir }

以直接地址为目的操作数

MOV Rn { A, dir, #data }

以 Rn 为目的操作数

MOV @Ri { A, dir, #data }

以间接地址为目的操作数

数据传送——内部存储器传送 MOV

例 1: R1=20H;（ 20H ） =55H; 指令 MOV A， @R1; 执行后，A=55H

例 2:（ 40H ） =30H; 指令 MOV R7， 40H; 执行后， R7=30H

例 3: MOV R7，＃ 40H; 执行后， R7=40H 。

- 例 判断下列指令的正误：

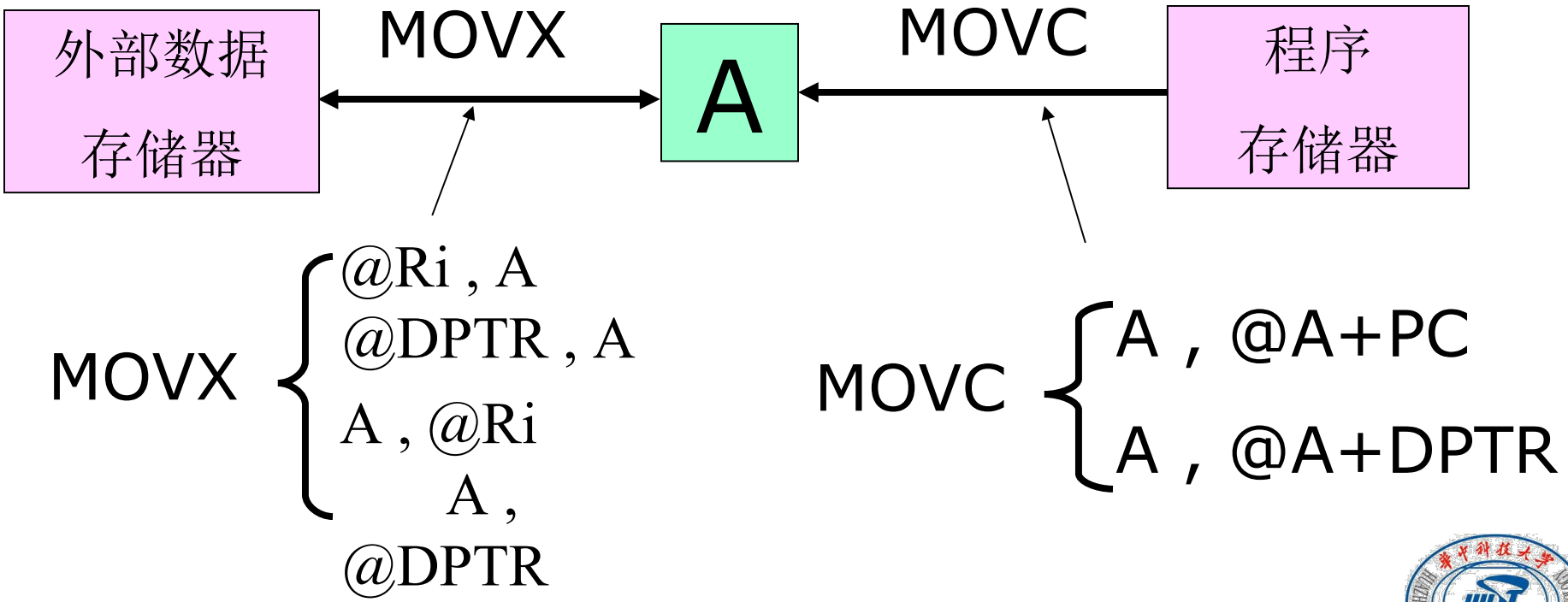
MOV 29H,R7	√	MOV 25H,P1	√
MOV 56H,#70H	√	MOV 34H,28H	√
MOV R3,R7	×	MOV @R3,R7	×
MOV R3,#D2H	√	MOV #34H,28H	×
MOV A,#280H	×	MOV P3,P1	√

- 编程将 R3 的内容送 R1 。

```
MOV A， R3
MOV R1， A
```

数据传送——外部存储器 MOVX、MOVC

- 外部数据存储器和程序存储器只能和 **A** 累加器进行数据传送，不能和内部 **RAM** 、 **SFR** 进行传送



数据传送——外部数据存储器传送 MOVX

MOVX 指令举例

例：实现片外数据存储器数据传送 **(2000H)→(2100H)**。

```
MOV   DPTR，  #2000H      ;   DPTR= 2000H
MOVX  A，  @DPTR          ;   A← X
MOV   DPTR，  #2100H      ;   DPTR= 2100H
MOVX  @DPTR，  A          ;   (2100H)← X
```

	片外 RAM	
	地址	内容
DPTR→	2000H	X
	…	
	2100H	

	外 RAM	
	地	容
	2000H	X
	…	
DPTR→	2100H	X

片外数据存储器不能直接寻址。下列为非法指令：

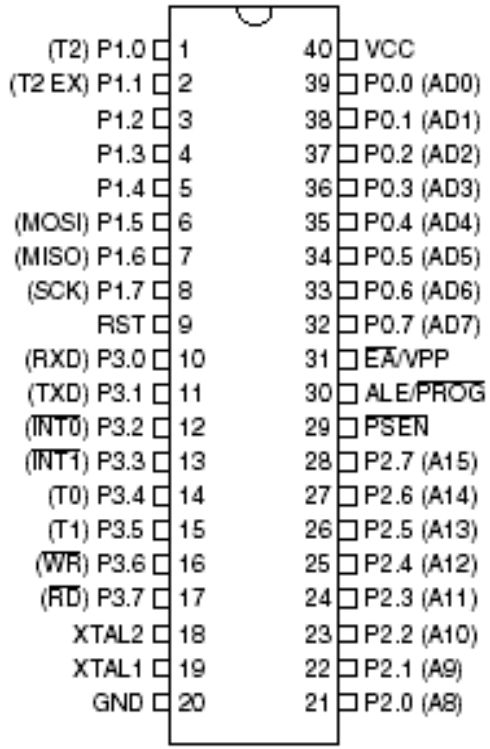
MOVX A，2000H ×

MOVX 2100H，2000H ×

数据传送——外部数据存储器传送 MOVX

- 使用 P2 口和 8 位寄存器 Ri 间接寻址：

```
MOV  P2，  #20H  ；高位地址
MOV  R0，  #00   ；低位地址
MOVX  A，  @R0          ；读片外 RA
MOV  P2，  #21H  ；改变高位地址
MOVX  @R0，  A          ；写片外 RA
```



数据传送——程序存储器传送 MOVC

- 查表指令 **MOVC**，多用于查常数表程序，直接求取常数表中的函数值
  - 只能从程序存储器读取数据到 **A** 累加器
  - 只能使用变址寻址方式
- **DPTR** 为基址寄存器，查表范围为 **64KB** 程序存储器任意空间

```
MOVC  A，  @A+DPTR      ；  A←(A+DPTR)
```

- **PC** 为基址寄存器，常数表只能在查表指令后 **256B** 范围内

```
MOVC  A，  @A+PC        ；  A← (A+PC)
```

数据传送——程序存储器传送 MOVC

例 查表法求 $Y=X^2$。设 $X(0\le X\le 15)$ 在片内 RAM 20H 单元，要求将查表求 Y，存入片内 RAM 21H 单元

1）用 DPTR 作基址寄存器实现

```
        ORG  0100H
SQU:    MOV  DPTR，  #TAB  ；确定表首地址
        MOV  A，  20H                ；取 X
        MOVC A，  @A+DPTR  ；查表求 Y=X²
        MOV  21H，  A                ；保存 Y
        RET                          ；子程序结束
          …                                ；其它程序段

        ORG  0200H                   ；常数表格首地址
TAB:    DB  00，  01，  04，  09，…，  225  ；平方表
```

数据传送——程序存储器传送 MOVC

例 查表法求 $Y=X^2$。设 X(0≤X≤15) 在片内 RAM 20H 单元，要求将查表求 Y，存入片内 RAM 21H 单元

1）用 PC 作基址寄存器实现

```
指令地址          源程序
                  ORG 0100H      ；程序起始地址
0100H        SQU: MOV A，20H          ；取 X
0102H             ADD A，#3     ；修正偏移量，取指令后 PC 指向下一条
0104H             MOVC A，@A+PC ；查表求 Y=X2
0105H             MOV 21H，A             ；存结果
0107H             RET                    ；子程序结束
0108H        TAB: DB 00，01，04          ；平方表
010BH             DB 09，…，225
```

```
        ORG 0200H                ；常数表格首地址
TAB:    DB 00，01，04，09，…，225 ；平方表
```

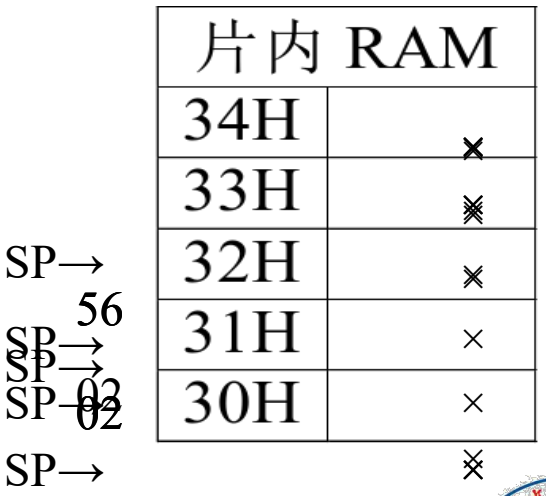
数据传送——堆栈操作指令

- 入栈指令： PUSH dir ； SP←SP+1，(SP)←(dir)
- 出栈指令： POP dir ； (dir)←(SP)，SP←SP-1

例：设A=02，B=56H，执行下列指令序列后，SP = ？ A = ？，B = ？

```
SBR：   MOV  SP，  #30H；设栈底
        PUSH  A          ；保护现场
        PUSH  B
        MOV   A，  #0
        MOV   B，  #01
              …
        POP   B；恢复现场
        POP   A
        RET
```

堆栈操作示意：



51 堆栈： 先加后压，先弹后减

数据传送——字节交换 XCH

- 字节交换指令实现 A 与 Rn 、 @Ri 、 dir 的数据双向传送

1. 字节交换指令

XCH A，Rn ；A←→Rn
XCH A，@Ri ；A←→(Ri)
XCH A，dir ；A ←→(dir)

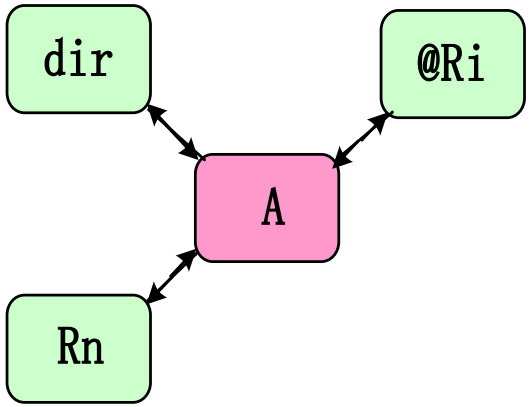


图2.8 XCH交换指令示意图

例 设 A= 29H，（2AH）＝ 38H，执行指令

执行指令“ XCH A， 2AH”后 ： A= 38H， (2AH)= 29H

习题 将片内 RAM 60H 单元与 61H 单元的数据交换

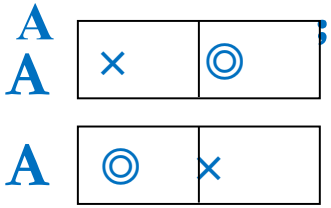
XCH 60H， 61H ← 对吗？

数据传送——半字节交换 XCHD

- XCHD： @Ri 与 A 的低 4 位交换
- SWAP： A 的高 4 位和低 4 位交换

XCHD A，@Ri ； $A_{0\sim3} \leftarrow (Ri)_{0\sim3}$

SWAP A ； $A_{4\sim7} \leftarrow A_{0\sim3}$

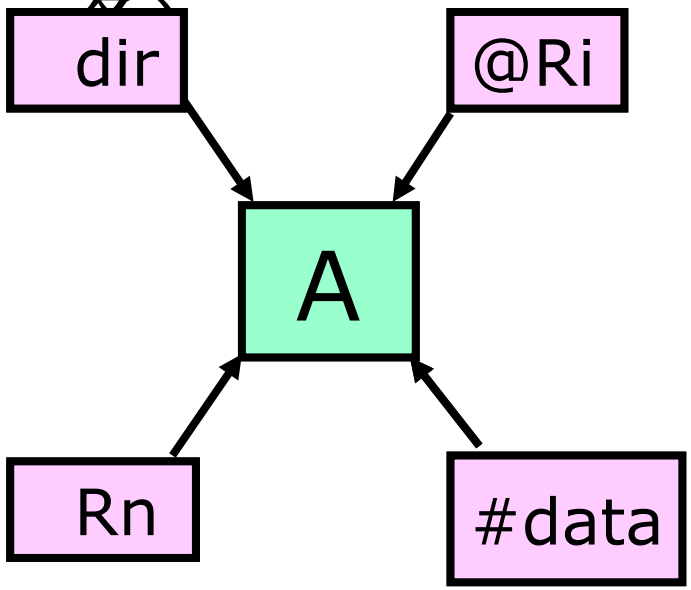


CPU 的指令系统

- 寻址方式
- 数据传送与交换指令
- 算术运算、逻辑运算指令
- 控制转移指令
- 位操作指令

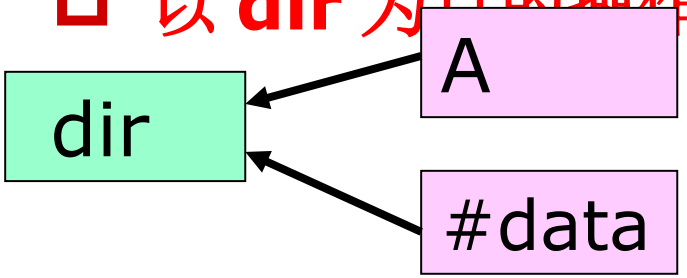
算数逻辑运算指令

- 以 **A** 为目的操作数的算术 / 逻辑运算指令（ **24** 条）



加	ADD	A，	Rn,
进位加	ADDC		@Ri
借位减	SUBB		#data
与	ANL		dir
或	ORL		
异或	XRL		

- 以 **dir** 为目的操作数逻辑运算指令（ **6** 条）



与	ANL	dir，	A
或	ORL		#data
异或	XRL		

算数逻辑运算指令

- 减指令只有带借位减，因此在多字节减法中，最低字节作减法时，注意先清 CY
- 逻辑运算是按位进行的，两数运算的运算法则：

与：有“0”则“0”；
或：有“1”则“1”
异或：同为“0”，异为“1”；与“0”异或值不变，与“1”异或值变反

- 逻辑指令常用于对数据位进行加工。

例： A=0FH, 执行 ORL A,#86H 后 A＝8FH
例： A=0FH, 执行 XRL A,#86H 后 A＝89H

算数逻辑运算指令——加 1、减 1 指令

加 1 指令

INC { A, Rn, @Ri, dir, DPTR }

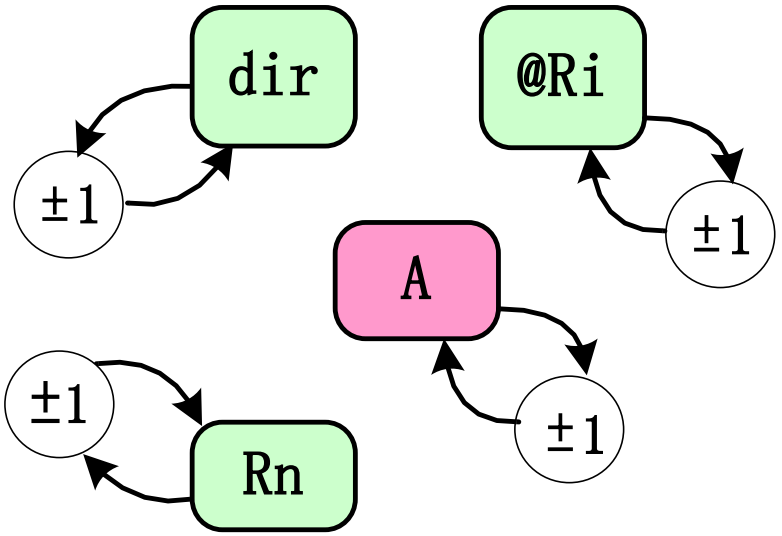


图2.13 INC、DEC指令

减 1 指令

DEC { A, Rn, @Ri, dir }

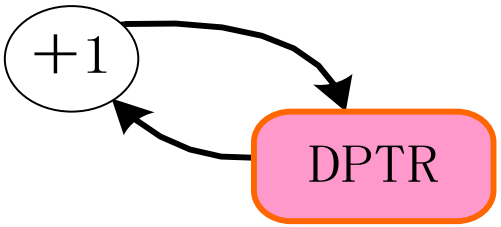


图2.14 INC DPTR

算数逻辑运算指令——十进制调整指令

- 计算机完成二进制加法其和也为二进制，如果是十进制相加（即 BCD 码相加）想得到十进制的结果，就必须进行十进制调整（即 BCD 码调整）
- DA A ；将 A 中二进制相加和调整成 BCD 码
- DA A 指令只对一个字节和调整，如为多字节相加必须进行多次调整。此指令不能对减法结果做调整

例： 完成 **56H+17H** 的编程

```
MOV A，  #56H  ；  A 存放 BCD 码 56H
MOV B，  #17H  ；  B 存放 BCD 码 17H
ADD A，   B       ；  A=6dH
DA A                ；  A=73H
SJMP $              ；   $ 代表当前的指令所在的地址
```

算数逻辑运算指令——关于 PSW 标志位

- 凡是对 A 操作指令（包括传送指令）都将 A 中 1 个的奇偶反映到 PSW 的 P 标志位上。即 A 中奇数个“ 1”， P=1 ；偶数个“ 1”， P=0 。
- 传送指令、加 1 、减 1 指令、逻辑运算指令不影响 Cy 、 OV 、 AC 标志位。
- 加减运算指令影响标志位，乘除指令使 Cy=0 ，当乘积大于 255 ，或除数为 0 时， OV 置 1 。
- 对进 / 借位 Cy （指令中用 C 表示）进行操作的指令和大环移指令，显然会影响 Cy 。
- 具体指令对标志位的影响可参阅附录 A 。
- 标志位的状态是控制转移指令的条件，因此指令对标志位的影响应该记住。

算数逻辑运算指令——关于 PSW 标志位

例： A= 9AH， R2= E3H， PSW= 0，执行指令

ADDC A， R2 后求：

A= 7DH ， Cy= 1 ， OV= 1 ； AC= 0 ， P= 0

PSW= 10000100 = 84H

```
  1001 1010
  1110 0011
+         0
-----------
1 0111 1101
```

CY：进 / 借位标志
AC：辅助进 / 借位标志
OV：溢出标志
P：奇偶标志

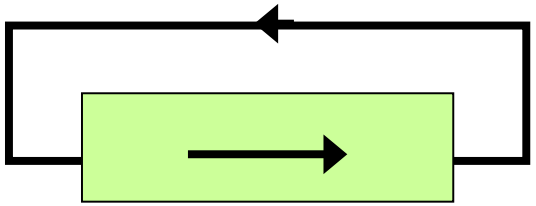
CY	AC	F0	RS1	RS0	OV	---	P
1	0	0	0	0	1	0	0

算数逻辑运算指令——移位指令

- 移位指令（仅对 **A** ）

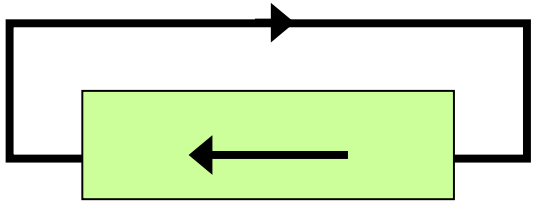
设 A 10010110 CY=1

RR A
右环移



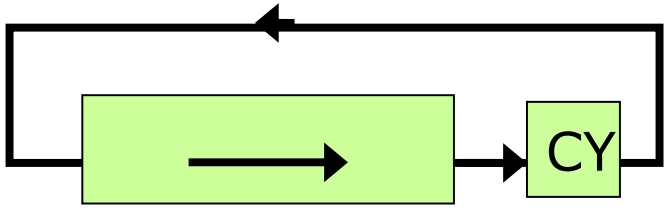
A 01001011

RL A
左环移



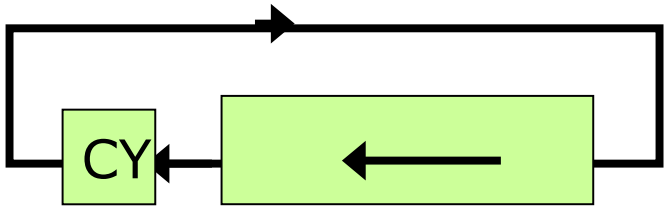
A 00101101

RRC A
右大环移



A 11001011 CY=0

RLC A
左大环移



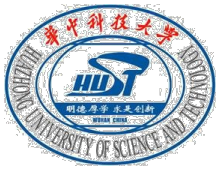
A 00101101 CY=1

CPU 的指令系统

- 寻址方式
- 数据传送与交换指令
- 算术运算、逻辑运算指令
- 控制转移指令
- 位操作指令

控制转移指令

- 改变指令的执行顺序，转到指定的 PC 地址执行
- **无条件转移**：无需判断，执行该指令就转移到目的地址
- **条件转移**：需判断标志位是否满足条件，满足条件转移到目的地址，否则顺序执行
- **绝对转移**：转移的目的地址用绝对地址指示，通常为无条件转移
- **相对转移**：转移的目的地址用相对于当前 PC 的偏差（偏移量）指示，通常为条件转移
- **长转移或长调用**： 64KB 地址范围内转移 / 调用
- **短转移或短调用**： 2KB 地址范围内转移 / 调用



控制转移指令——程序调用和返回类指令

- 子程序长调用： LCALL addrl16 ；addr16→PC

说明：

（1）该指令功能是

①保护断点，即当前 **PC**（本指令的下一条指令的首地址）压入堆栈。

②子程序的入口地址 **addr16** 送 **PC**，转子程序执行。

（2）本指令为 **64KB** 地址范围内的调子程序指令，子程序可在 **64KB** 地址空间的任一处。

（3）本指令的机器码为三字节。

- 子程序短调用： ACALL addr11 ;addr11→PC

说明：

（1）该指令的功能是

①保护断点，即当前 **PC** 压入堆栈。

② **addrl11→PC0 ～ 10,** 而 **PC11 ～ 15** 保持原值不变。

（2）本指令为 **2KB** 地址范围的调子程序指令，子程序入口距当前 **PC** 不得超过 **2KB** 地址范围。

（3）本指令的机器码为二字节。

控制转移指令——程序调用和返回类指令

- 子程序返回指令： RET ；从调用子程序返回
- 功能：从栈顶弹出断点到 PC

- 对比中断返回指令： RETI ；从中断服务程序返回
- 功能：从栈顶弹出断点到 PC，并恢复中断优先级状态触发器。

控制转移指令——绝对转移指令

- **短转移**： AJMP addr11 ;addr11→PC0～10

说明：
①转移范围：本指令为 **2KB** 地址范围内的转移指令。对转移目的地址的要求与 **ACALL** 指令对子程序入口地址的要求相同。
②机器码形式：本指令为 **2** 字节指令。

- **长转移**： LJMP addr16 ； addr16→PC0～15

说明：
①本指令为 **64KB** 程序存储空间的全范围转移指令。转移地址可为 **16** 位地址中的任意值。
②本指令为 **3** 字节指令

- **间接转移**： JMP @A+DPTR ； A+DPTR→PC

例 **A=02H**，**DPTR=2000H**，指令 **JMP @A+DPTR** 执行后，**PC=2002H** 。也就是说，程序转移到 **2002H** 地址单元去执行。

控制转移指令——绝对转移指令

例　现有一段程序如下：

```
        MOV   DPTR，  #TABLE
        JMP   @A+DPTR
TABLE：   AJMP   PROC0
        AJMP   PROC1
        AJMP   PROC2
        AJMP   PROC2
```

根据 JMP　@A+DPTR 指令的操作可知，

当 A=00H 时，程序转入到地址　PROC0　处执行；

当 A=02H 时，转到 PROC1 处执行……

可见这是一段多路转移程序，进入的路数由 A 确定。因为 AJMP 指令是 2 字节指令，所以　A　必须为偶数。

控制转移指令——相对转移指令

- 无条件相对转移

SJMP　rel　　；PC+rel→PC，

即 As ＋ 2 ＋ rel→PC，机器码为 80 rel

说明：

As 为源地址（本指令的首地址），该指令为 2 字节指令，执行本指令时

当前 PC ＝ As+2，　rel　为转移的偏移量，转移可以向前转（目的地址小于源地址），也可以向后转（目的地址大于源地址），因此偏移量 rel 是 1 字节有符号数，用补码表示（－ 128 ～＋ 127），所以指令转移范围在离源地址 As 的－ 126 ～＋ 129 字节之间

控制转移指令——条件转移指令

□ 累加器为零（非零）转移

JZ rel ； **A=0** 则转移（**As+2+rel**）→**PC**， **A≠0** 程序顺序执行，机器码为 **60rel**

JNZ rel ； **A≠0** 则转移（**As+2+rel**）→**PC**， **A =0** 程序顺序执行，机器码为 **60rel**

□ 减 1 不等于零转移

DJNZ Rn , rel ； Rn-1 →Rn, Rn≠0, （As+2+rel）→ PC

DJNZ dir , rel ； (dir)-1 →(dir), (dir)≠0, （As+3+rel）→ PC

□ 比较转移指令

CJNE A,dir , rel ;A≠(dir), （As+3+rel）→ PC

CJNE A , #data , rel ;A≠#data, （As+3+rel）→ PC

CJNE Rn , #data , rel ;Rn≠#data, （As+3+rel）→ PC

CJNE @Ri , #data , rel ;(Ri)≠#data, （As+3+rel）→ PC

控制转移指令——条件转移指令

分析程序运行后 **A** 中的结果：

```
        MOV   23H，  #0AH
        CLR    A
LOOP：  ADD   A，  23H
        DJNZ   23H，  LOOP
        SJMP   $
```

根据程序可知

A=10+9+8+7+6+5+4+3+2+1=55=37H

控制转移指令——条件转移指令

例 编写程序，要求读 **P1** 端口上的信息，若不为 **55H**，则程序等待，直到 **P1** 端口为 **55H** 时，程序才往下顺序执行。

```
MOV   A，#55H    ；A＝55H
CJNE  A，P1，$   ；P1≠55H，则程序循环执行本指令
...
```

在实际编程中，转移的目的地址不管是 **addr11**、**addr16**、还是 **rel**，均是以符号地址表示的（如 **SJMP ABC**， **AJMP LOOP…**），转移的类型是通过指令的操作符来决定的。

CPU 的指令系统

- 寻址方式
- 数据传送与交换指令
- 算术运算、逻辑运算指令
- 控制转移指令
- 位操作指令

控制转移指令——位操作指令

- 在程序中位地址的表达有多种方式：
  - 用直接位地址表示，如 D4H 。
  - 用“ .”操作符号表示，如 PSW.4 ，或 D4H.4
  - 用位名称表示，如 RS1 。
  - 用用户自定义名表示。如 ABC BIT D4H ，其中 ABC 定义为 D4H 位的位名， BIT 为位定义伪指令。
  - 以上各例均表示 PSW.4 的 RS1 位。
- 位操作类指令的对象是 C 和直接位地址，由于 C 是位累加器，所以位的逻辑运算指令目的操作数只能是 C

控制转移指令——位操作指令

- 位清零

```
CLR   C      ;  0→CY
CLR   bit    ;  0→bit
```

- 位置 1

```
SETB   C     ;  1→CY
SETB   bit   ;   1→bit
```

- 位取反

```
CPL  C     ;   /CY  →CY
CPL  bit   ;   /bit → bit
```

控制转移指令——位操作指令

- 位与

```
ANL  C，bit   ；CY ∧（bit）→CY
ANL  C，/bit  ；CY∧（#bit）→CY
```

- 位或

```
ORL  C，bit   ；CY∨（bit）→CY
ORL  C，/bit  ；CY∨（#bit）→CY
```

- 位传送

```
MOV  C，bit   ；（bit）→CY
MOV  bit, C   ；CY→bit
```

控制转移指令——位操作指令

- 位转移：根据位的值决定转移，均为相对转移指令，设 **As** 为下面各指令的首地址

JC rel ;CY=1，则转移（As+2+rel→PC），否则顺序执行
JNC rel ;CY=0, 则转移（As+2+rel→PC），否则顺序执行
JB bit，rel ;(bit)=1, 则转移（As+3+rel→PC），否则顺序执行
JNB bit,rel ;(bit)=0, 则转移（As+3+rel→PC），否则顺序执行
JBC bit,rel ;(bit)=1, 则转移（As+3+rel→PC），且该位清零；否则
顺序执行

控制转移指令——位操作指令

例：用位操作指令实现 X＝X0⊕X1，设X0为P1.0， X1为P1.1， X为ACC.0。

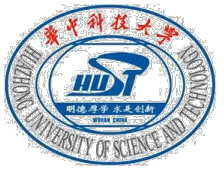
解：因位操作指令中无异或指令，依据X＝X0⊕X1＝X0(#X1)＋X0(#X1) 用与、或指令完成，编程如下：

```
X   BIT  ACC.0
X0  BIT  P1.0
X1  BIT  P1.1      ;位定义
MOV  C，X0
ANL  C，  /X1      ;C=X0∧#X1
MOV  20H，C      ;暂存于20H 单元
MOV  C，X1
ANL  C，/X0      ;C=X0∧#X1
ORL  C，20H       ；C＝X0X1＋X0X1
MOV  X，C
SJMP  $
```

51 单片机指令系统小结

- 51 系列单片机指令系统中，不同的存储空间寻址方式不同，适用的指令不同，必须进行区分。
- 指令是程序设计的基础，掌握指令的功能，操作的对象和结果，对标志位的影响，应要求熟记。

8051 单片机汇编程序设计



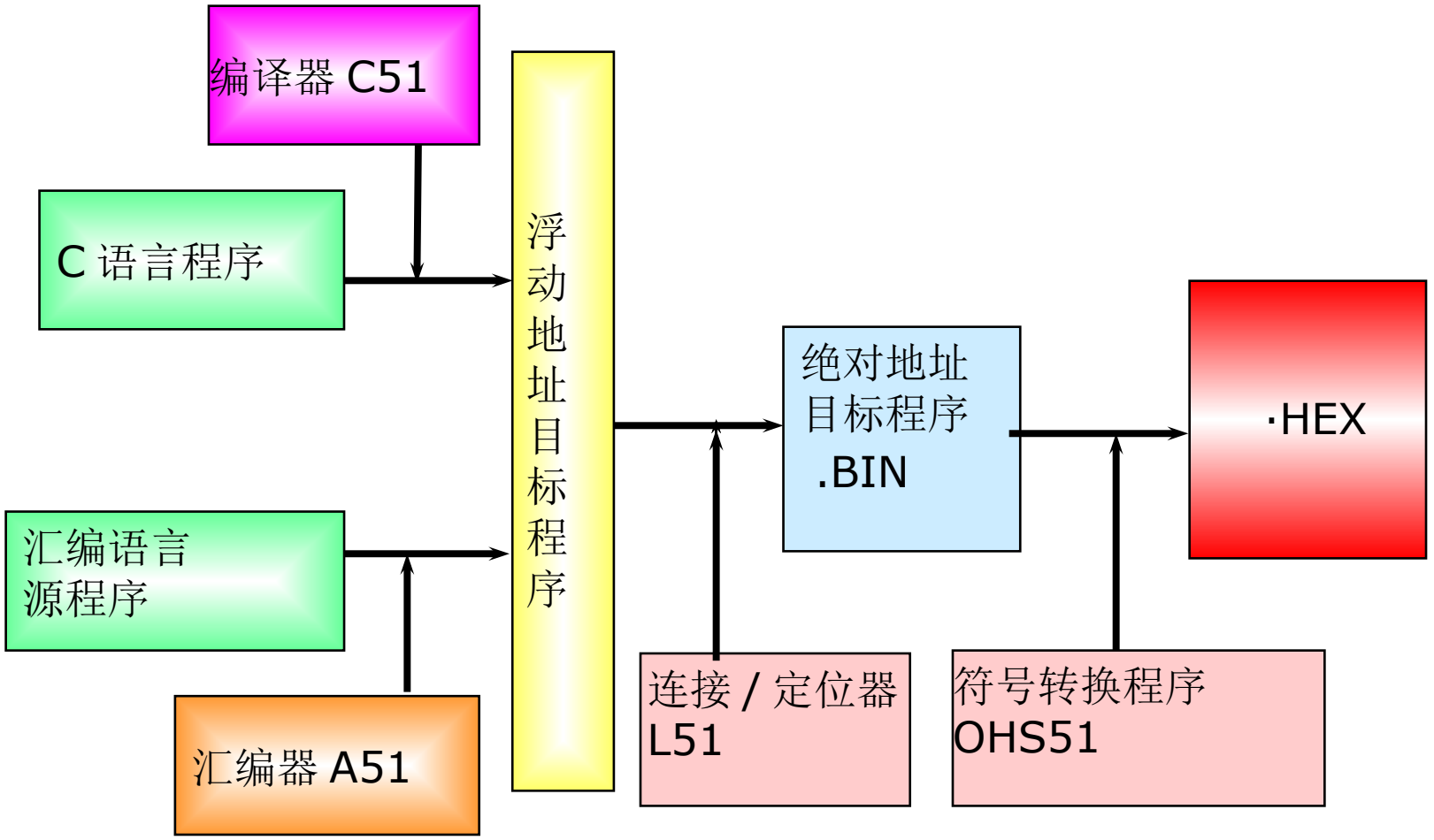
51 单片机汇编程序设计

- 概述
- 伪指令
- 汇编语言程序设计步骤
- 顺序程序设计
- 分支程序设计
- 循环程序设计
- 位操作程序设计
- 子程序

概述

- MCS-51 单片机编程可以用汇编也可以是高级语言（如 C 语言）
- 高级语言编程快捷，但程序长，占用存储空间大，执行慢
- 汇编语言产生的目标程序简短，占用存储空间小，执行快
- 无论是高级语言还是汇编语言，源程序都要转换成目标程序（机器语言）单片机才能执行
- 软件集成开发环境（IDE）：集成编辑器、汇编器、编译器、连接 / 定位器、符号转换程序等软件工具

概述


编译器 C51
C 语言程序
汇编语言
源程序
汇编器 A51
浮动地址目标程序
连接 / 定位器
L51
绝对地址目标程序
.BIN
符号转换程序
OHS51
·HEX

概述

- 汇编程序的汇编过程：
  - 确定程序中每条汇编语言指令的指令机器码
  - 确定每条指令在存储器中的存放地址
  - 提供错误信息
  - 提供目标执行文件 **(*.OBJ/*.HEX)** 和列表文件 **(*.LST)**

地址	机器码	源程序
		ORG 2000H
2000H	78 30	MAIN： MOV R0，#30H
2002H	E6	MOV A，@R0
		…

汇编程序设计——伪指令

- 机器指令：指令系统中的全部指令。每条机器指令都有对应的机器代码，可以被 CPU 执行。
- 伪指令： 汇编控制指令，没有指令代码，只用于汇编过程，为汇编程序提供汇编信息。

汇编程序设计——伪指令

□ 起始指令　　ORG nn

功能：定义程序或数据块的起始地址。指示此语句后面的程序或数据块以 nn 为起始地址，连续存放在程序存储器中。

指令地址	机器码	源程序
		ORG 2000H
2000H	78 30	MAIN： MOV R0， #30H
2002H	E6	MOV A， @R0
		…
		ORG 3000H
3000H	23	TAB： DB 23H， 100， ‘A’
3001H	64	
3002H	41	

汇编程序设计——伪指令

- 字节定义　标号：　DB （字节常数或字符或表达式）

功能：指示在程序存储器中以标号为起始地址的单元里存放的数为字节数据（八位二进制数）。
例如　LN：DB 32，'C'，25H，-1；
LN ～ LN+2　地址单元依次存放 20H，43H，25H，FFH

地址	数据
LN	20
LN+1	43
LN+2	25
LN+3	FF

- 字定义　标号：DW （字常数或表达式）

作用：　指示在程序存储器中以标号为起始地址的单元里存放的数为字数据（即 16 位的二进制数），
例如：
GH ： DW　1234H，5678H，08

GH	12
	34
GH+2	56
	78
GH+4	00
	08

汇编程序设计——伪指令

□ 保留字节　　标号： DS （数值表达式）

作用： 指示在程序存储器中保留以标号为起始地址的若干字节单元，其单元个数由数值表达式指定。

例如　L1 ： DS 32 ; 从 L1 地址开始保留 32 个存储单元。

□ 等值指令　　标号　EQU （数值表达式）

表示 **EQU** 两边的量等值，用于为标号或标识符赋值。例如：

```
        X1      EQU   2000H
        X2      EQU   0FH
                ...
MAIN:   MOV  DPTR , #X1 ;  DPTR=2000H
        ADD   A , #X2       ;  A=A+0FH
```

汇编程序设计——伪指令

- 位定义　标号　**BIT**　[位地址]

  作用：　同 **EQU** 指令，不过定义的是位操作地址。
  例如　　**AIC BIT P1.1**

- 汇编结束　**END**

  作用：　指示源程序段结束。

  END 指令放在程序的最后。

- **A51** 汇编程序还有一些其它的伪指令，列在教材表 **3-1** 中，以备查阅

汇编程序设计——宏指令

- 宏汇编功能：将需要反复多次执行的程序段定义成一个宏指令名（宏定义）。编程时，可在程序中使用宏指令名来替代被定义的程序段（宏调用）

宏定义过程：

宏指令名　MACRO　形式参数

…　　；被定义的程序段

ENDM

宏调用过程：

…

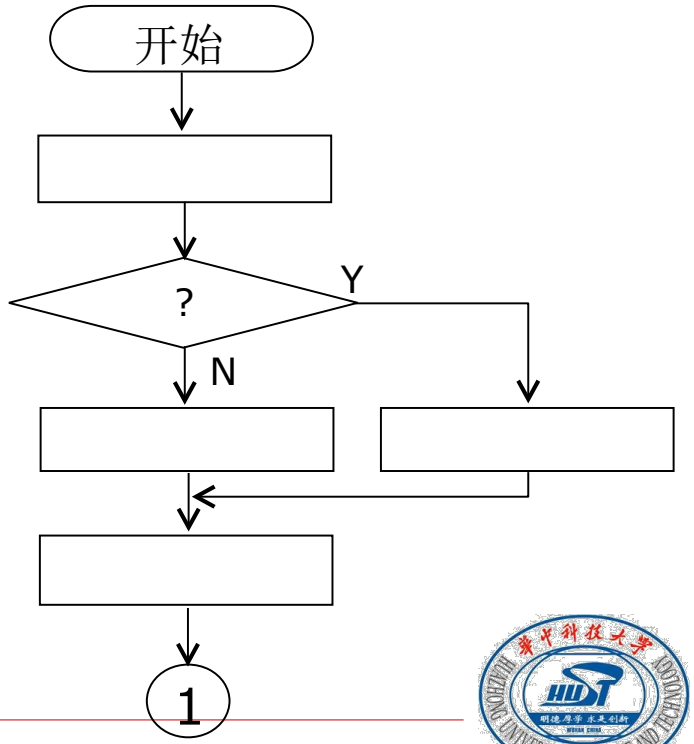
宏指令名　实际参数

…

宏指令名　实际参数

汇编程序设计——设计步骤

- 确定方案和计算方法
- 了解应用系统的硬件配置、性能指标
- 建立系统数学模型，确定控制算法和操作步骤
- 合理分配存储器单元和了解 I/O 接口地址
- 编制源程序
  - 按功能设计程序，明确各程序之间的相互关系
  - 用流程图表示程序结构和功能
  - 程序中用注释说明指令在程序中的作用，方便阅读、调试和修改



汇编程序设计——常见程序结构

- 顺序程序
- 分支程序
- 循环程序
- 子程序

汇编程序设计——顺序程序

- 顺序程序（简单程序），程序走向只有一条路径

例：变量在片内 RAM 的 20H 单元，其值在 00H~05H 之间，设计查表程序，查出变量的平方值，并放入片内 RAM 的 21[illegible]单元

```
ORG  0000H
     MOV  DPTR,#TAB
     MOV A,20H
     MOVC A,@A+DPTR ; 查表
     MOV 21H,A
     SJMP $
TAB2: DB 00H,01H,04H,09H,16H,25H
     END
```

地址	数据
TAB	0
TAB+1	1
TAB+2	4
TAB+3	9
TAB+4	16H
TAB+5	25H

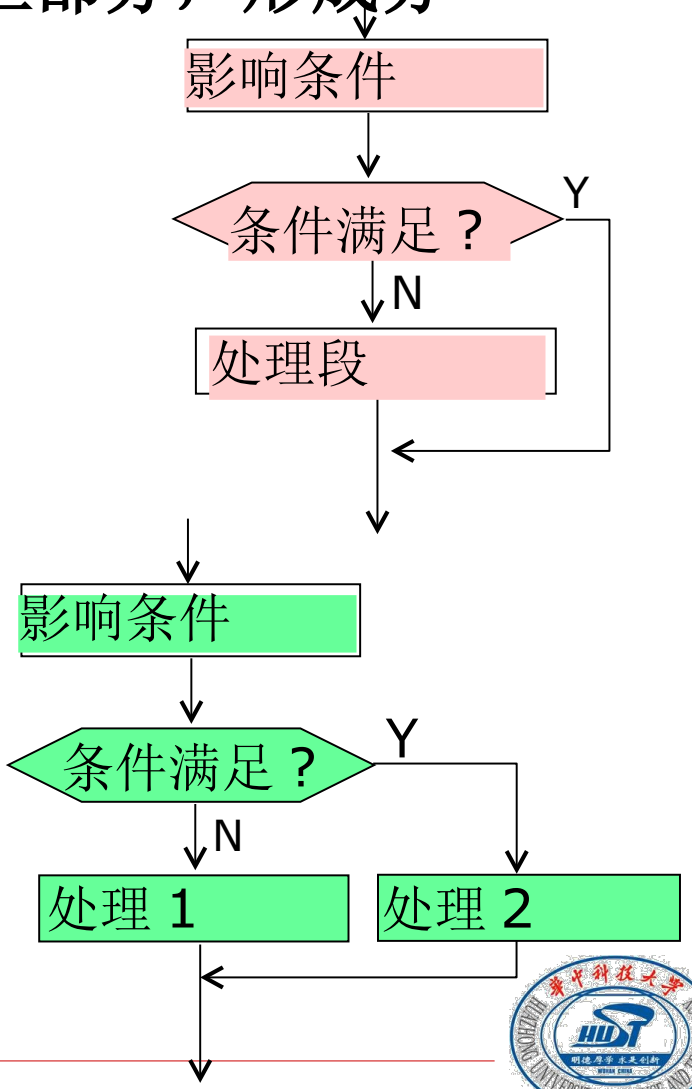
DPTR　　A

汇编程序设计——分支程序

- 由条件转移指令构成程序判断框部分，形成分支结构

例 1　求 8 位有符号数的补码

```
        ORG  0000H
      MOV A ，  R2
        JNB   ACC.7 ，  N    ；为正数？
        CPL   A              ；负数变补
        INC   A
        ORL     A,  #80H;  恢复负号
N ：    SJMP  $              ；结束
```



汇编程序设计——循环程序

- 当程序中的某些指令需要反复执行多次时，采用循环程序的方式，这样会使程序缩短，节省存储单元（并不节省执行时间）

例 **3-9** 设计一个延时 **10ms** 的延时子程序，已知单片机使用的晶振为 **6MHz** 。

分析 ：

延时时间与两个因素有关：晶振频率和循环次数。由于晶振采用 **6MHz** ，一个机器周期是 **2μs** ，用单循环可以实现 **1ms** 延时，外循环 **10** 次即可达 **10ms** 延时。

内循环如何实现 **1ms** 延时呢，程序中可先以未知数 **MT** 代替，从附录 **A** 查每条指令机器周期计算程序的执行时间。

汇编程序设计——循环程序

```
        ORG 0020H
DELAY:  MOV  R0, #0AH   ; 外循环 10 次
DL2:    MOV  R1, #7DH   ; 内循环 MT 次
DL1:    NOP
        NOP             ; 空操作指令
        DJNZ R1, DL1
        DJNZ R0, DL2
        RET
```



内循环 **DL1** 到指令 **DJNZ R1， DL1** 的计算：

（1+1+2）×2μS×MT=1000μS=1mS

若需要延时更长时间，可以采用多重循环。

汇编程序设计——子程序

- 当一段程序需多次应用，或为多人应用时，可将其设计为子程序
- 通过 **ACALL** 和 **LCALL** 两条指令调用子程序
- 子程序节省存储单元，使程序简短、清晰
- 子程序必须以 **RET** 指令结尾
- 两个注意事项：
  - 保护现场。用堆栈保护使用了某些存储单元或寄存器
  - 设置入口参数和出口参数。用存储单元或寄存器传递参数

汇编程序设计——子程序

例： delay 是延时子程序

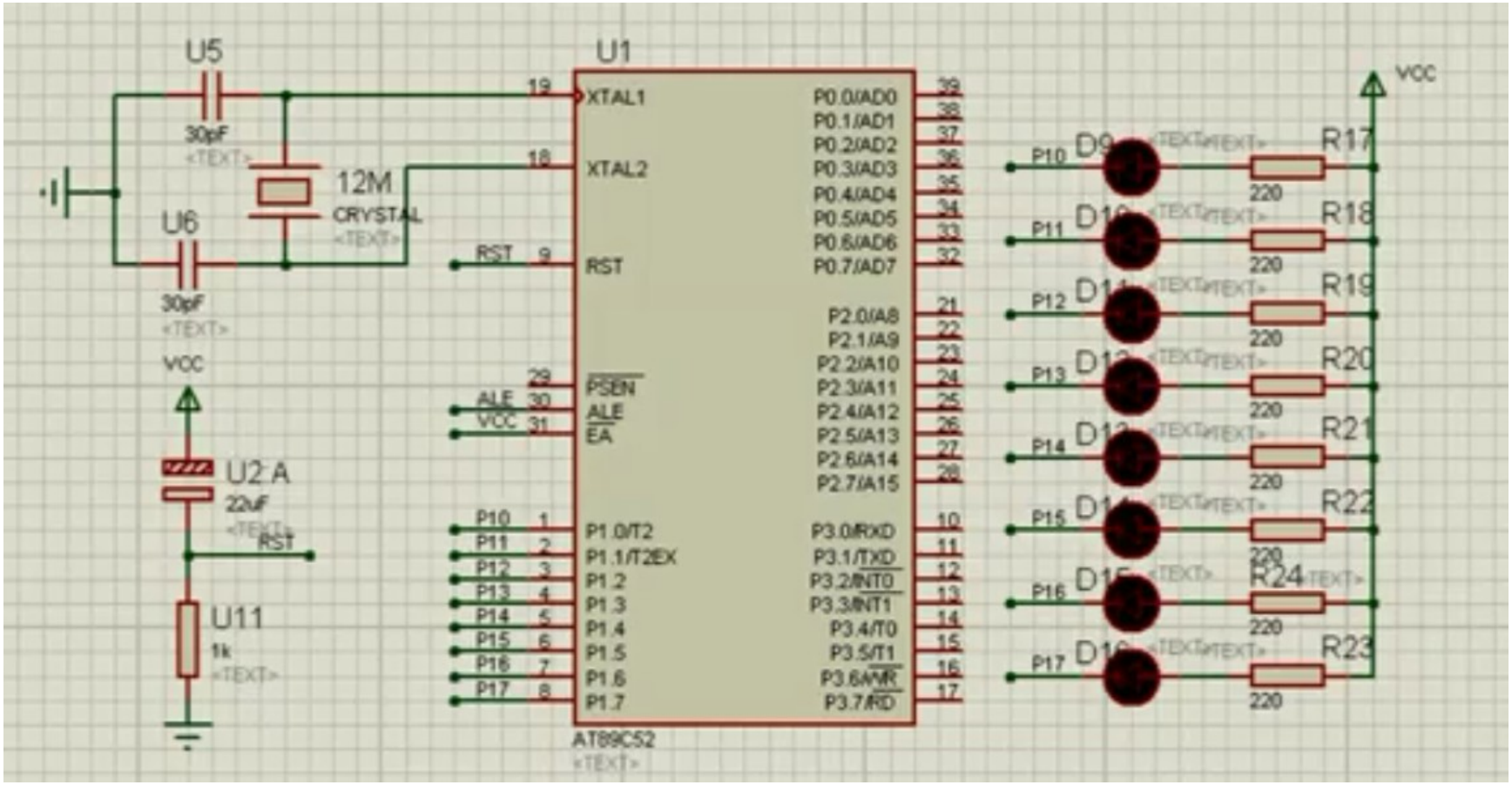
```
        POWER EQU P0.7
        ORG 0000H
        SJMP MAIN
        ORG 0030H
MAIN:   CLR POWER
START:  MOV P1,#0FEH
        ACALL DELAY
        MOV P1,#0FFH
        ACALL DELAY
        AJMP START
DELAY:  MOV R4,#2
D3:     MOV R2,#250
D1:     MOV R3,#229
D2:     DJNZ R3,D2
        DJNZ R2, D1
        DJNZ R4,D3
        RET
        END
```

汇编程序设计——小结

- 程序设计的关键在于指令熟悉和算法（思路）正确、清晰，对复杂的程序应先画出流程图。只有多做练习多上机调试，熟能生巧，才能编出高质量的程序。
- 伪指令是非执行指令，提供汇编程序以汇编信息，应正确使用。
- 掌握顺序程序、分支程序、循环程序、子程序等各类程序的设计方法，并能熟练应用查表技术简化程序的设计

试验

- 用 **Proteus** 设计电路图，理解：复位、时钟、并口



试验

- 用汇编语言设计 **P1** 口八个 **LED** 的跑马灯，先用 **keil** 仿真，没问题再下载到 **Proteus** 模拟电路板测试

